6GGW / NetSwitch — Technical Specification

One-page-per-subsystem spec for NetSwitch and every tool in this repo. For the authoritative math numbering see `MATH-INDEX.md` (M1–M40); for the module tree see `SOURCE-MANIFEST.md`; for the AI + maths inventory see `AI-AND-MATHS.md`. Every module is single-file C++17 (T-SQL for the data plane), builds on Linux (`g++`) and Windows (MinGW `-static`), and ships its own `selftest`.

Status legend: **RUN** = measured/working now · **BACKEND** = engine done, needs real device/credentials · **STUB** = honest hook awaiting your code.

---

1. NetSwitch switch core

The product is a phone-to-phone / thin-client **switch**: it registers devices, replicates media streams between them, meters call quality, and reroutes over the least-cost path with region failover. Asymmetric envelope by design — **1300 kbps down / 250 kbps up / 32 kbps audio floor**.

Module	Program	Interface	I/O	Status
`server-cpp/`	ggw_server	HTTP + socket control surface	session registry; least-cost reroute `cost=RTT·(1+load/100)` (M35); geodistance `d=0.0001607·RTT` (M36); region failover	RUN
`replicate/`	ggw_replicate	phone↔phone transport	STATE (reliable delta), FILE (SHA-256 verified, M37), AUDIO/VIDEO (real-time)	RUN
`stream-ctl/`	ggw_streamctl	routing/rerouting	least-cost path, POP select, RKF45 battery ODE (M24/M25), pipe capacity (M26), split at 1/2/7/15/40/59/80% (M27)	RUN
`stream-qc/`	ggw_streamqc	quality metering	PSNR (M22), blockiness (M23) → Black/Grey/White class → Low/Med/High reroute trigger	RUN
`voice-edge/`	ggw_voice_edge	walkie-talkie relay	G.711 μ-law (M20), RTP/RFC-3550 pack (M21)	RUN
`optimize/`	ggw_optimize	bitrate allocator	AUTO fills the down/up envelope, priority floor first, 2:1 split; MANUAL clamps + reroute verdict	RUN
`registrar/`	ggw_registrar	device/session registration	CIDR prefix→mask (M39), one-time code issue	RUN
`secure/`	ggw_secure	transport security	TLS 1.3 tunnel + cert gen (M40, OpenSSL)	BACKEND (certs/keys)
`backup-codes/`	ggw_backupcodes	recovery codes	Crockford base32 CSPRNG, 50-bit (M38)	RUN
`client-cli/`	ggw_cli	thin-client CLI	switch client commands	RUN
`client-qt/`	Qt app	desktop GUI	claim/dashboard/distance/rerouting panels	BACKEND (Qt build)

---

2. Benchmark / measurement suite

AI2ORBIT CO. Network system Pat. pend. Founded in 2018. Copyright 2018-2027. All rights reserved.

Module	Program	Measures	Status
`report-engine/`	ggw_report	MFLOPS (matmul+FMA), CPU/RAM/disk inventory, OS-compat verdict → phone HTML	RUN
`dramm/`	ggw_dramm	DRAMM tight kernel (M1) vs naive kernel — honest same-machine speedup; CUDA twin `.cu`	RUN (+CUDA STUB)
`loopbench/`	ggw_loopbench	sustained ops/s, FPS, MFLOPS, closed-form `C=11.267648…` (M2) with checksums	RUN
`cputimer/`	ggw_cputimer, ggw_radiosig	RDTSC-calibrated MHz (M31), radio-signal reduce (M32)	RUN
`diskbench/`	ggw_diskbench	fsync-honest IOPS / MB-s / latency percentiles (M33)	RUN
`thermal/`	ggw_thermal	per-part temp vs trip, throttle-swing learn (M34)	RUN (real sensors = BACKEND)
`sysmon/`	ggw_sysmon	live cores/mem/load, per-core Gops (M29), FPS (M30) — no personal data	RUN

3. HW-test tools (the "testing tools" from the AI2Orbit primer)

Module	Program	What it times / tests	Status
`intbench/`	ggw_intbench	INT2/4/8/16 quantized MAC loopset, three bars GPU/CPU/HPU	RUN (CPU) · GPU/HPU STUB
`drammtune/`	ggw_drammtune	**NEW** — CPU/GPU combination search (threads × duty) to run DRAMM fastest + battery savings	RUN (CPU) · GPU/energy STUB→BACKEND
`hwtools/`	ggw_memtimer	reserve-memory-per-calc timer + CLI log-list + dX (today/tomorrow) compare	RUN
`hwtools/`	ggw_nettrace	40 pings + 15 traceroutes + 11 pingroutes concurrently, per-type timing	RUN

4. Numeric / signal functions

Module	Program	Computes	Status
`ddtm/`	ggw_ddtm	8×8 DCT-II/IDCT (M6/M7), powered/unpowered split (M8), entropy (M9), PSNR + best scaler (M10/M11), MHz banding (M12)	RUN
`ipr/`	ggw_ipr	DCT image-block transform, depth ratioing (M28)	RUN
`approve/`	ggw_approve	deterministic "server AI == thin-client AI": workload (M3), result hash (M4), approval ratio (M5)	RUN

5. Data plane (MS SQL)

AI2ORBIT CO. Network system Pat. pend. Founded in 2018. Copyright 2018-2027. All rights reserved.

Module	File	What it is	Status
`netswitch-sql/`	netswitch_sql.cpp	ODBC connector: device_session, sccm_block, scaler_result	BACKEND (target instance)
`netswitch-sql/scaling/`	ggw_scaling.sql	T-SQL scaling math (M13–M19), DDTM constant −0.090908889 (M13), Python cross-check	RUN (SQL) · locks with T1/T2/LogLogRad

6. Build & verify conventions

```
./build-all.sh                       # Linux: build + selftest every module
g++ -std=c++17 -O2 <mod>.cpp -o <mod>
x86_64-w64-mingw32-g++ -std=c++17 -O2 <mod>.cpp -o <mod>.exe -static
```

Every tool: single-file, zero hidden deps, deterministic where a checksum is claimed, a `selftest` sub-command, and cross-platform checksum identity where the maths is deterministic. Measured vs. modelled vs. stub is labelled in-tool and in each README — nothing invented.

7. Pending / awaiting your input (kept explicit, nothing dropped)

- **GPU code** → wires `gpu_loopset()` (intbench) and `gpu_throughput()` (drammtune) live.
- **Real energy sensor** (RAPL/battery API) → replaces the drammtune battery model with a watt reading.
- **Thermal engine** — the two thermals (Justacon/Entropia, van der Waals) need one worked example each (inputs → the number you already get) before I lock the formulas; not guessed.
- **DDTM constant** locks once T1/T2/LogLogRad values are given.
- **MS SQL target instance** (2025/2026) for the data-plane connector.
- **Phone model + Android/iOS version** for the NDK/Capacitor build.

AI2ORBIT CO. Network system Pat. pend. Founded in 2018. Copyright 2018-2027. All rights reserved.